



```
▼ TERMINAL
```

```
pick 6c3a9ca first commit
pick 7ed487c Delete .DS_Store
pick 8c63f73 Update config.py
pick 281e292 Update config.py
pick 5196d38 Update config.py
pick 17744ee added aws cred
pick aab2ba7 added secret manager code to access credentials

# Rebase aab2b0f onto d4258d0 (7 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                       commit's log message, unless -C is used, in which case
#                       keep only this commit's message; -C is same as -C but
#                       opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (Continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#     create a merge commit using the original merge commit's
#     message (or the oneline, if no original merge commit was
#     specified); use -C <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#     to this position in the new commits. The <ref> is
#     updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
# However, if you remove everything, the rebase will be aborted.
```



Introducing Today's Project!

In this project, I will; - Identify how a web app insecurely stores credentials. - See how GitHub's secret scanning feature can block insecure code from being pushed to a repository. - Update the web app to use AWS Secrets Manager to store and retrieve credentials securely. - Verify that secured web app code can be made public without exposing sensitive credentials.

Tools and concepts

Services I used were; AWS Secrets Manager, AWS IAM, GitHub Key concepts I learnt include; - Applying security best practices for managing credentials in code - Understanding GitHub's secret scanning feature and why it's important - Using AWS Secrets Manager to securely store and manage AWS credentials - Retrieving secrets programmatically in Python using the AWS SDK (boto3) - Cleaning up sensitive data from Git commit history by rebasing and handling merge conflicts

Project reflection

This project took me approximately 1.5 hours to complete. The most challenging part was resolving the merge conflict during the Git rebase, especially making sure I kept the secure version of the code and removed all traces of the hardcoded credentials. It was most rewarding to successfully integrate AWS Secrets Manager into the application and confirm that the sensitive data was completely removed from the repository's history, leaving behind a clean, secure, and professional setup.



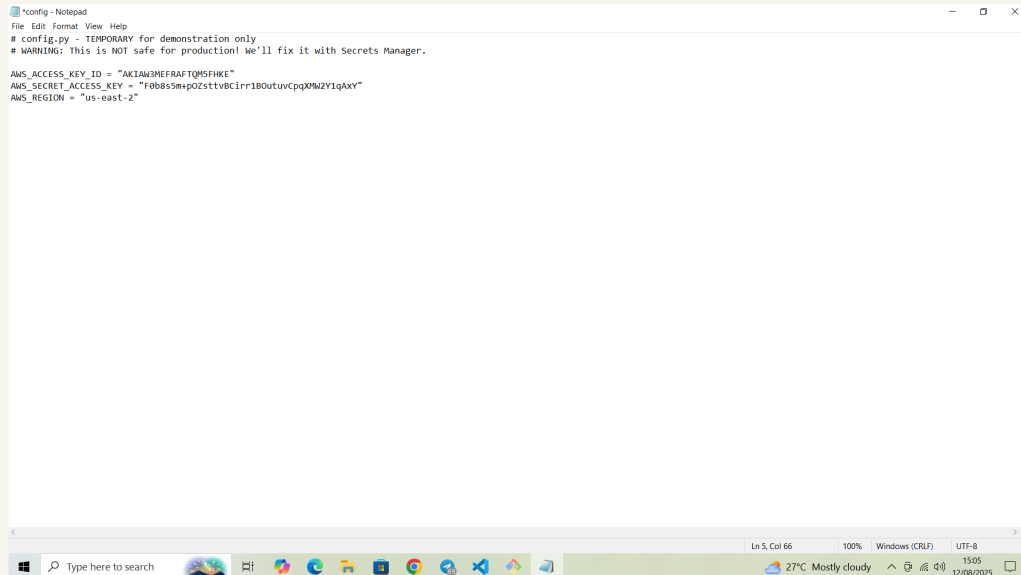
I did this project as it is an essential project for a cloud engineering career



Hardcoding credentials

In this project, a sample web app is exposing my AWS credentials publicly. It is unsafe to harcode credentials because once someone else gets access to my credentials, they can use them to access my AWS account, delete resources, steal data, and cause damage.

I've set up the initial configuration with Access keys. These credentials are just examples because I am going to imagine that these are real AWS credentials that would give someone access to my AWS account!



```
*config - Notepad
File Edit Format View Help
# config.py - TEMPORARY for demonstration only
# WARNING: This is NOT safe for production! We'll fix it with Secrets Manager.

AWS_ACCESS_KEY_ID = "AKIAJ3MEFRAFQHSFIKE"
AWS_SECRET_ACCESS_KEY = "F0b85m+p0ZsttvBCirr1B0utuvCpQ0%GZ1q4xy"
AWS_REGION = "us-east-2"
```




Using my own AWS credentials

As an extension for this project, I also decided to run the web app locally by using my own AWS credentials; I set up my virtual environment and installed packages such as boto3, fast-api and uvicorn. I'll need these packages because my web app's main file (app.py) will be leveraging them to connect to my AWS environment.

When I first ran the app, I ran into an error because the AWS credentials I was using weren't valid. Unfortunately I'm using placeholder credentials in my web app's config.py file, which means they're not real and I'll have to create my own credentials in my AWS account to ensure the functionality of my web app.

To resolve the 'InvalidAccessKeyId' error, I updated my web app's configuration file (config.py) so it would use the new credentials I created in my AWS account, which were the Access Key ID, Secret Access Key and the Region. I verified that the region I chose was also where my S3 buckets would be stored as it's best practice to keep resources in the same region.



```
{
  "buckets": [
    "nickbhanster-foodrecipes",
    "taekwondo-career"
  ]
}
```



Pushing Insecure Code to GitHub

Once I updated the web app code with credentials, I forked the repository because I want to showcase my version of the web app, and make changes to it without affecting the original repository. A fork is different from a clone in the sense that When you fork a repository, you're making a copy of it in your GitHub account online. If your forked repository is public, anyone can see it! But When you clone a repository, you're creating a local, offline copy. By default, no one can see it since it's stored on your computer.

To connect my local repository to the forked repository, I ran the command `git remote set-url origin`. This updates the remote origin to point to my fork instead of the original repository. Then I used `git add` and `git commit` to save the changes I made to `config.py`. Finally, `git push` uploads those committed changes to the remote origin, in this case, my forked repository on GitHub

GitHub blocked my push because its secret scanning feature detected that I was hardcoding AWS credentials into the code. This is a good security feature because it helps prevent sensitive information like access keys from being exposed in public repositories, which could lead to unauthorised access, data breaches, or unexpected charges.



```
Command Prompt
C:\Users\BORIS\Documents\network-security-secretsmanager>git commit -m "Updated config.py with hardcoded credentials"
[main 5607d1b] Updated config.py with hardcoded credentials
1 file changed, 3 insertions(+), 2 deletions(-)

C:\Users\BORIS\Documents\network-security-secretsmanager>git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 407 bytes | 407.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote:
remote: - GITLAB PUSH PROTECTION
remote:
remote:   Resolve the following violations before pushing again
remote:
remote:   - Push cannot contain secrets
remote:
remote:   (?) Learn how to resolve a blocked push
remote:   https://docs.github.com/code-security/secret-scanning/working-with-secret-scanning-and-push-protection/working-with-push-protection-from-the-command-line#resolving-a-blocked-push
remote:
remote:
remote:   -- Amazon AWS Access Key ID --
remote:   locations:
remote:     - commit: 5607d1b847d1d15aee808075065e7737c1054
remote:       path: config.py:4
remote:
remote:   (?) To push, remove secret from commit(s) or follow this URL to allow the secret.
remote:   https://github.com/dev-boris07/network-security-secretsmanager/security/secret-scanning/unblock-secret/325d08e123a0b09101ac1d110q
remote:
remote:   -- Amazon AWS Secret Access Key --
remote:   locations:
remote:     - commit: 5607d1b847d1d15aee808075065e7737c1054
remote:       path: config.py:5
remote:
remote:   (?) To push, remove secret from commit(s) or follow this URL to allow the secret.
remote:   https://github.com/dev-boris07/network-security-secretsmanager/security/secret-scanning/unblock-secret/325d08e123a0b09101ac1d110q
remote:
remote:
remote: To https://github.com/dev-boris07/network-security-secretsmanager.git
+ https://github.com/dev-boris07/network-security-secretsmanager.git
error: failed to push some refs to 'https://github.com/dev-boris07/network-security-secretsmanager.git'

C:\Users\BORIS\Documents\network-security-secretsmanager>
```

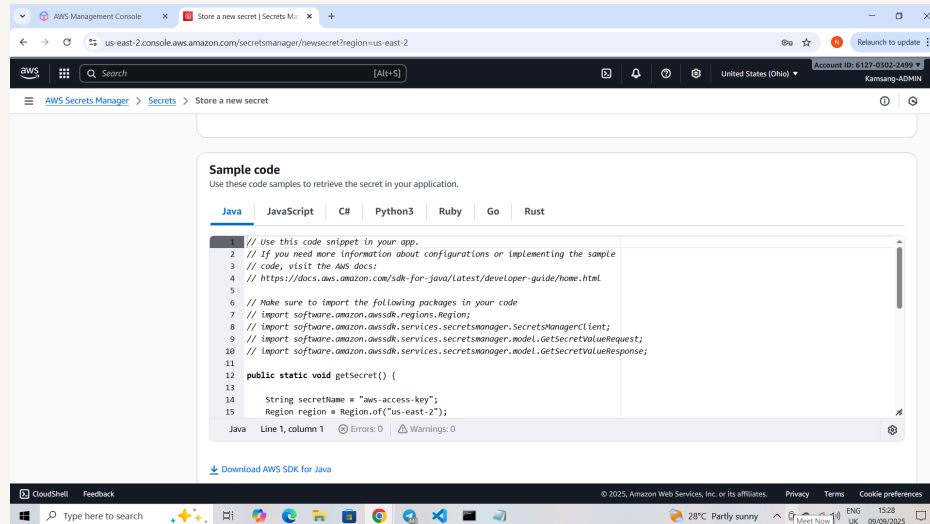


Secrets Manager

Secrets Manager is a secure service provided by AWS for storing and managing sensitive information like API keys, database passwords, and other secrets. I'm using it to store the AWS credentials that would otherwise be hardcoded in config.py. Other common use cases include managing credentials for databases, third-party services, or internal APIs, and automatically rotating those secrets to enhance security.

Another feature in Secrets Manager is secret rotation, which means automatically updating and replacing your secrets on a regular schedule without manual intervention. It's useful in situations where you're managing high-risk credentials like database passwords, API keys, or service account credentials.

Secrets Manager provides sample code in various languages, like Python, Java, and Node.js. This is helpful because it shows developers exactly how to retrieve secrets from Secrets Manager in their applications. By using this sample code, developers can avoid hardcoding sensitive information and instead fetch it securely at runtime, reducing the risk of credential leaks and improving overall application security.





Updating the web app code

I updated the config.py file to retrieve AWS credentials securely from Secrets Manager instead of hardcoding them. The get_secret() function will connect to AWS Secrets Manager using the boto3 library, fetch the secret named "aws-access-key" from the specified region (like "us-east-1"), and return the credentials stored there. This function includes error handling using a try...except block to catch issues like missing permissions or incorrect secret names. By doing this, the application now retrieves sensitive information at runtime, improving security and following best practices for managing secrets.

I also added code to config.py to extract the individual credential values from the secret returned by the get_secret() function. This is important because our application expects specific variables, i.e. AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_REGION, to be defined so it can authenticate with AWS services. The new code calls get_secret(), parses the returned JSON using json.loads(), and assigns the appropriate values to each variable. This ensures the app gets the credentials securely at runtime instead of relying on hardcoded values.



```
config - Notepad
File Edit Format View Help
import boto3
from botocore.exceptions import ClientError

def get_secret():
    secret_name = "aws-access-key"
    region_name = "us-east-2"

    # Create a Secrets Manager client
    session = boto3.session.Session()
    client = session.client(
        service_name='secretsmanager',
        region_name=region_name
    )

    try:
        get_secret_value_response = client.get_secret_value(
            SecretId=secret_name
        )
    except ClientError as e:
        # For a list of exceptions thrown, see
        # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
        raise e

    secret = get_secret_value_response['SecretString']

    # Your code goes here.
```




Rebasing the repository

Git rebasing is the process of rewriting commit history by moving or editing commits to create a cleaner, more linear sequence. I used it to remove a specific commit that accidentally included sensitive AWS credentials. This was necessary because pushing secrets to a repository, even temporarily, poses a serious security risk. By using interactive rebase and the drop command, I was able to erase that commit from the history as if it never existed, helping ensure the credentials weren't exposed or retrievable later.

A merge conflict occurred during rebasing because the commit I removed contained hardcoded AWS credentials, and the following commit also modified the same file, `config.py`. Git wasn't sure how to reconcile the changes once the earlier commit was dropped. I resolved the merge conflict by manually editing the file: I removed the hardcoded credentials and kept the updated code that uses AWS Secrets Manager to securely retrieve the credentials. I also deleted all the merge conflict markers (`<<<<<<`, `====`, and `>>>>>>`) to clean up the file. This ensured the file only contained the secure version of the code and allowed the rebase to continue successfully.

Once the merge conflict was resolved, I verified that my hardcoded credentials were removed by checking the commit history and the `config.py` file in my forked GitHub repository. I made sure that the file now only contains the secure version that uses AWS Secrets Manager, and that no previous commits include the sensitive credentials I had accidentally added. This confirms that the rebase and cleanup were successful, and that my repository is now safe from accidentally exposing secrets through Git history.



```
Command Prompt - git rebase -i --root
git commit --amend # first commit
git commit --amend # update config.py
git commit --amend # update config.py
git commit --amend # update config.py
git commit --amend # update config.py with hardcoded credentials
git commit --amend # update config.py with secrets manager credentials

git rebase Bab5b44 onto e76c393 (-? commands)

# Commands
# 1. pick (commit) = use commit
# 2. reword (commit) = use commit, but edit the commit message
# 3. edit (commit) = use commit, but stop for editing
# 4. squash (commit) = use commit, but meld into previous commit
# 5. fixup [-C | -c] (commit) = like "squash" but keep only the previous
#    commit's message, unless -C is used, in which case
#    keep only this commit's message; -C is same as -c but
#    opens the editor
# 6. exec (command) = run command (the rest of the line) using shell
# 7. break = stop here (continue rebase later with "git rebase --continue")
# 8. drop (commit) = remove commit
# 9. label (label) = label current HEAD with a name
# 10. reset (label) = reset HEAD to a label
# 11. merge [-C (commit) | -c (commit)] (label) [4 newline]
#    create a merge commit using the original merge-commit's
#    message (or the online, if no original merge-commit was
#    specified); use -C (commit) to reuse the commit message
# 12. undo-ref (ref) = track a predecessor (or the ref) to be restored
#    to this position in the new commit. The ref is
#    updated at the end of the rebase

These lines can be re-ordered; they are executed from top to bottom.
If you remove a line here THAT COMMIT WILL BE LOST.
However, if you remove everything, the rebase will be aborted.
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

